CM #2
Listes avancées

TI301 – P2 2025/2026

Nicolas FLASQUE

Objectifs :

- Comprendre le rôle des pointeurs dans les listes.
- Visualiser et appliquer les insertions de cellule dans des cas particuliers.
- Garder la cohérence de listes avancée
- Optimiser leurs algorithmes pour traiter les données le plus efficacement possible.
- Implémenter en C les différents traitements abordés

Autres types de liste

Liste avec un **pointeur repérant la première cellule** et **un pointeur repérant la dernière cellule**

Quel avantage ? Pouvoir ajouter en fin de liste facilement

Liste dont **la dernière cellule pointe sur la première** (liste circulaire)

Quel avantage ? Parcourir la liste de manière circulaire, pour représenter des phénomènes cycliques

Liste dont **chaque cellule pointe la suivante et la précédente**

Quel avantage ? On peut la parcourir dans les deux sens, ce qui simplifie les traitements, mais oblige à bien 'raccrocher' les cellules

Liste avec un pointeur repérant la première et la dernière cellule

Principe : repérer immédiatement la dernière cellule plutôt que de parcourir toute la liste

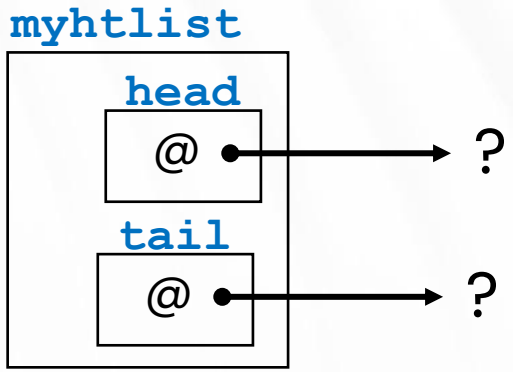
On gagne du temps de calcul
On utilise un peu plus de mémoire pour stocker le pointeur vers la fin de liste

Sur le même principe que la liste simple, on crée la structure suivante.

```
typedef struct s_ht_list
{
    t_cell *head;    // adresse de la première cellule
    t_cell *tail;    // adresse de la dernière cellule
} t_ht_list;
```

‘ht’ signifie ‘head and tail’

Visualisation et initialisation



Définition d'une variable de type t_ht_list :

```
t_ht_list myhtlist;
```

Pour l'initialiser en tant que liste vide, les deux pointeurs doivent être mis à **NULL**

```
myhtlist.head = NULL;
myhtlist.tail = NULL;
```

On créera la fonction `t_ht_list createHtList()` qui effectuera la tâche : crée une liste 'ht' vide bien initialisée – vue en exercice

Question : d'après son prototype, comment va-t-on l'utiliser ?

```
t_ht_list myhtlist;
```

Parcours, chaînage, insertion, suppression

Parcours (pour affichage par exemple)

Question : à partir de quel pointeur, `head` ou `tail`, peut-on parcourir une `t_ht_list` ?

Réponse : à partir de head comme pour une liste simple. On ne peut aller que dans un sens

Chaînage

Question : où peut-on faire des chaînages (ajout de cellules) dans une `t_ht_list` ?

Réponse : en tête, comme pour une liste 'standard' et en queue, à partir du pointeur tail

Suppression : éléments vus en cours, programme complet à voir en TD / TP

Insertion dans une t_ht_list

Pour illustrer en même temps le parcours et le chaînage, vus pour les listes simples, nous allons détailler la méthode d'insertion.

On dispose d'une `t_ht_list` (potentiellement vide) dont les éléments sont classés par ordre croissant.

On souhaite y ajouter une nouvelle cellule (avec une certaine valeur) de manière à ce que les éléments de la liste soient toujours bien triés.

On distingue 3 cas :

- La liste est vide
- La liste n'est pas vide et on doit insérer la nouvelle cellule avant la fin
- La liste n'est pas vide et on doit insérer la nouvelle cellule à la fin

Insertion dans une t_ht_list

<u>Premier point :</u>

Comment repérer qu'une liste est vide ?

Si les pointeurs `head` et `tail` valent NULL

On suppose qu'on dispose d'une fonction `int isEmpty(t_ht_list);` pour ce test

Deuxième point :
comment savoir où insérer la nouvelle cellule ? (dans la liste triée)

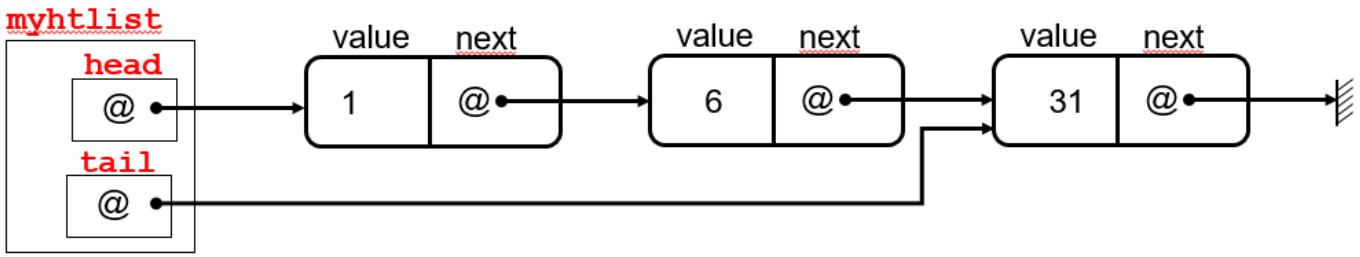
Tant que sa valeur est plus grande que la valeur de la cellule courante

Comment savoir si on doit insérer dans la liste ou à la fin ?

Si on a parcouru toute la liste sans trouver de valeur plus grande, on doit insérer à la fin

Illustration

Soit la liste suivante :



Nous devons la parcourir avec 2 pointeurs :

`temp` pour la cellule courante
`prev` pour la cellule précédant la cellule courante

Pourquoi ?

Exemple : insertion de la cellule pointée par **newcell**

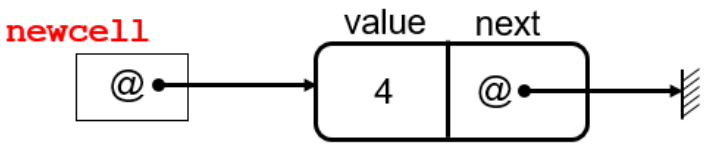
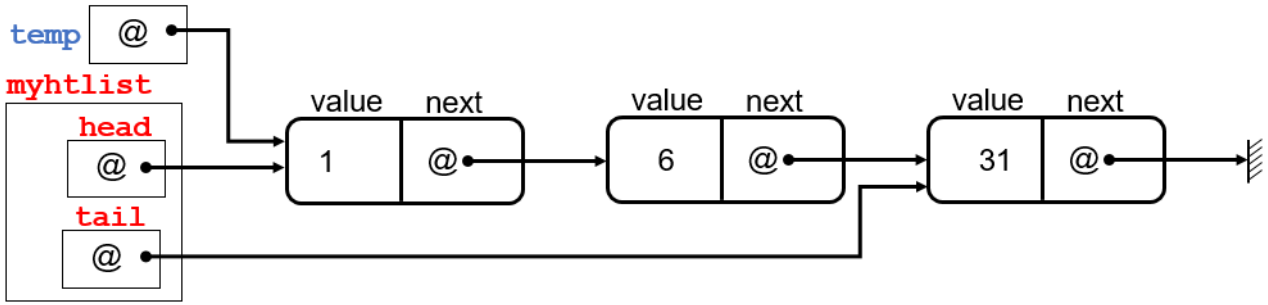
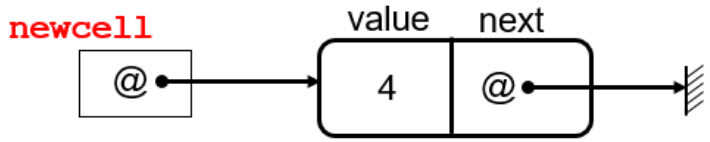


Illustration - 2

Initialisation le pointeur `t_cell *temp` en tête de liste

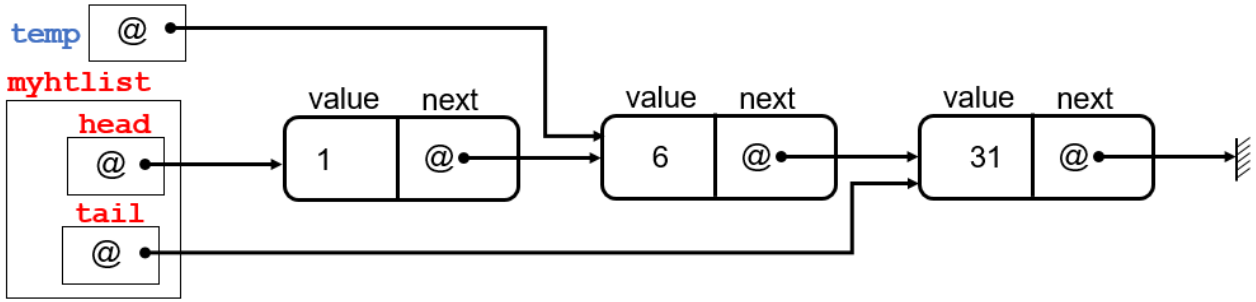
`temp = myhtlist.head;`





On ne peut pas insérer newcell ici, car 4 est plus grand que 1
Donc, avançons dans la liste

`temp = temp->next;`



4 est plus petit que 6, donc il faut insérer `newcell` **avant** `temp`

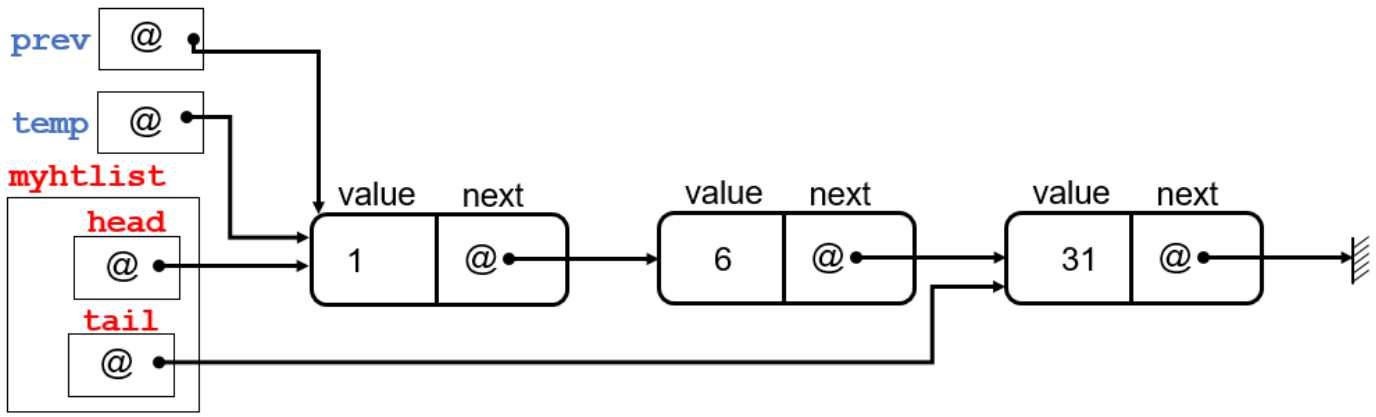
Mais on ne peut pas revenir en arrière !

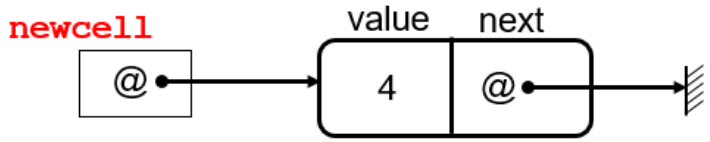
Il faut se 'souvenir' de la cellule précédente

Illustration - 3

On ajoute un pointeur `t_cell *prev;` initialisé en tête de liste

```
temp = myhtlist.head;
prev = temp;
```





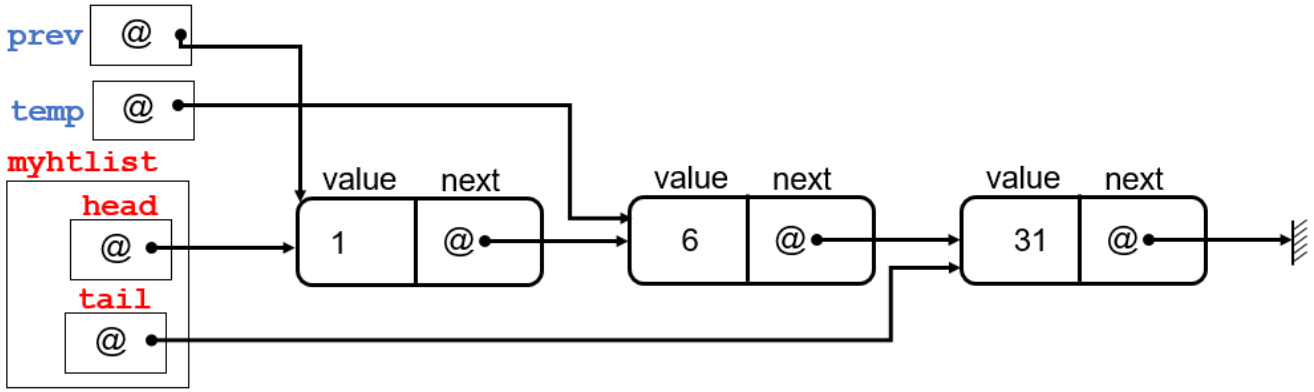
On ne peut pas insérer newcell ici, car 4 est plus grand que 1
Donc, avançons dans la liste

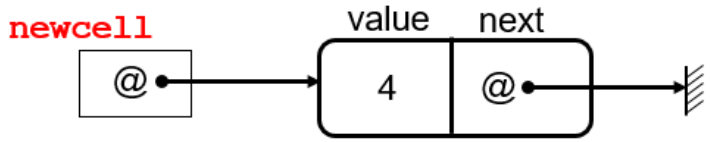
```
// on 'sauvegarde' temp avant de le 'déplacer'
prev = temp;
temp = temp->next;
```

Illustration - 4

...Donc, avançons dans la liste

```
// on 'sauvegarde' temp avant de le 'déplacer'
prev = temp;
temp = temp->next;
```





La suivante de `prev` devient `newcell`
La suivante de `newcell` est `temp`

```
prev->next = newcell;
newcell->next = temp;
```

Synthèse : fonction C

Commençons à écrire la fonction `insertOrderedHtList`

Son rôle est d'insérer, à la bonne place, une nouvelle cellule avec la valeur donnée en deuxième paramètre, dans la liste pointée par le premier paramètre.

```
void insertOrderedHtList(t_ht_list *ptr_list, int val)
{
    t_cell *newcell, *prev, *temp;
    newcell = createCell(val);

    // cas numéro 1 : la liste est vide
    if (isEmptyHtList(*ptr_list) == 1)
    {
       ptr_list->head = newcell;
       ptr_list->tail = newcell;
    }
```

Rappel : on doit utiliser un **pointeur** sur `t_ht_list`

car l'insertion peut modifier `'head'` et/ou `'tail'`

Synthèse : fonction (2)

Sinon, on parcourt la liste :

```
else
{
    temp = ptr_list->head;
    prev = temp;

    while((temp != NULL) && (temp->value < newcell->value))
    {
        prev = temp;
        temp = temp->next;
    }
    prev->next = newcell;
    newcell->next = temp;

    if (temp == NULL)
    {
        ptr_list->tail = newcell;
    }
}
```

```
void insertOrderedHtList(t_ht_list *ptr_list, int val)
{
    t_cell *newcell, *prev, *temp;
    newcell = createCell(val);

    // cas numéro 1 : la liste est vide
    if (isEmptyHtList(*ptr_list) == 1)
    {
        ptr_list->head = newcell;
        ptr_list->tail = newcell;
    }
```

Synthèse : fonction (3)

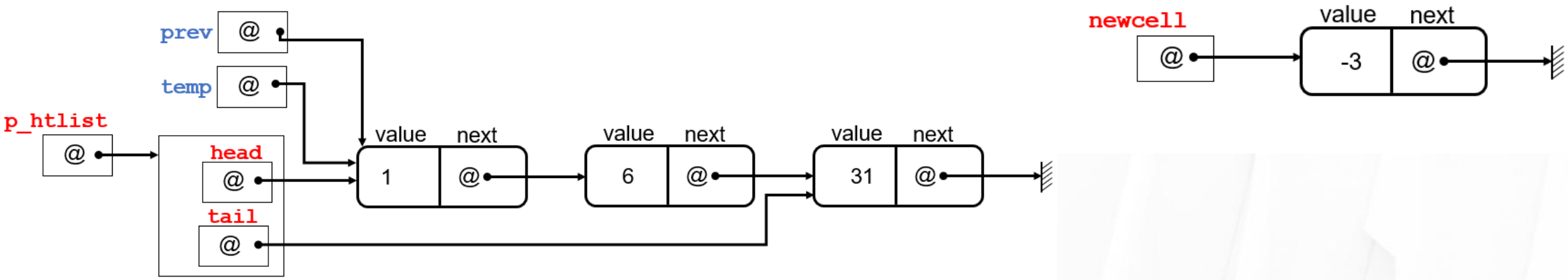
<u>Il reste un cas particulier pour lequel ça ne fonctionne pas :</u>

Quand on doit insérer en tête, c'est-à-dire si on n'a pas besoin de déplacer `temp`

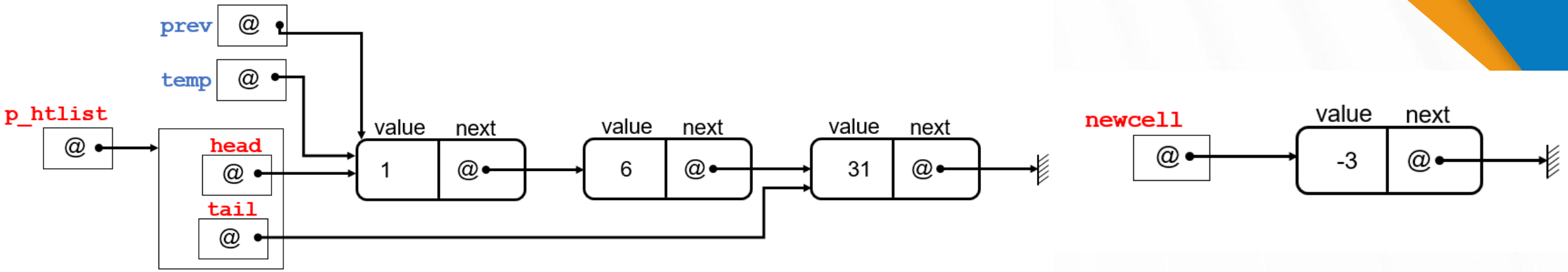
Par exemple, insérons une cellule avec la valeur -3 : elle doit venir en tête de liste

Dans ce cas, on doit mettre `newcell` en premier , et donc on modifie le champ 'head' de la liste

Dans quel état se trouvent les variables quand `temp` n'a pas été modifié ?



Synthèse : fonction (4)



Comment repérer ce cas ?

Tester si `prev == temp` ou si `temp == ptr_list->head`

Que faire ? Insérer en tête :

```
newcell->next = ptr_list->head;
ptr_list->head = newcell;
```

Visualisation avec C Tutor

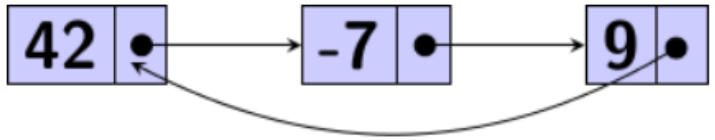
https://pythontutor.com/render.html#code=%23include%20%3Cstdlib.h%3E%0Atypedef%20struct%20s_cell%20t_cell%3B%0A%0Astruct%20s_cell%7B%0A%20%20%20%20int%20value%3B%0A%20%20%20%20t_cell%20*next%3B%0A%7D%3B%0A%0Atypedef%20struct%20s_ht_list%0A%7B%0A%20%20t_cell%20*head,%20*tail%3B%0A%7D%20t_ht_list%3B%0A%0At_cell%20*createCell%28int%20val%29%0A%7B%0A%20%20t_cell%20*nouv%20%3D%20%28t_cell%20*%29malloc%28sizeof%28t_cell%29%29%3B%0A%20%20nouv-%3Evalue%20%3D%20val,%0A%20%20nouv-%3Enext%20%3D%20NULL%3B%0A%20%20return%20nouv%3B%0A%7D%0A%0Aint%20isEmptyHtList%28t_ht_list%20aList%29%0A%7B%0A%20%20return%20%28aList.head%20%3D%3D%20NULL%29%3B%0A%7D%0A%0Avoid%20insertOrderedHtList%28t_ht_list%20*p_htlist,%20int%20val%29%0A%7B%0A%20%20t_cell%20*newcell,%20*temp,%20*prev%3B%0A%20%20newcell%20%3D%20createCell%28val%29%3B%0A%20%0A%20%20if%20%28isEmptyHtList%28*p_htlist%29%29%0A%20%20%7B%0A%20%20%20%20p_htlist-%3Ehead%20%3D%20p_htlist-%3Etail%20%3D%20newcell%3B%20%20%0A%20%20%7D%0A%20%20else%0A%20%20%7B%0A%20%20%20%20temp%20%3D%20p_htlist-%3Ehead%3B%0A%20%20%20%20prev%20%3D%20temp%3B%0A%20%20%20%20%0A%20%20%20%20while%20%28%28temp%20!%3DNULL%29%26%26%28temp-%3Evalue%20%3C%20newcell-%3Evalue%29%29%0A%20%20%20%20%7B%0A%20%20%20%20%20%20prev%20%3D%20temp%3B%0A%20%20%20%20%20%20temp%20%3D%20temp-%3Enext%3B%0A%20%20%20%20%7D%0A%20%20%20%20%0A%20%20%20%20if%20%28prev%20%3D%3D%20temp%29%0A%20%20%20%20%7B%0A%20%20%20%20%20%20newcell-%3Enext%20%3D%20p_htlist-%3Ehead,%0A%20%20%20%20%20%20p_htlist-%3Ehead%20%3D%20newcell%3B%0A%20%20%20%20%7D%0A%20%20%20%20else%0A%20%20%20%20%7B%0A%20%20%20%20%20%20prev-%3Enext%20%3D%20newcell%3B%0A%20%20%20%20%20%20newcell-%3Enext%20%3D%20temp%3B%0A%20%20%20%20%20%20if%20%28temp%3D%3DNULL%29%0A%20%20%20%20%20%20%7B%0A%20%20%20%20%20%20%20%20p_htlist-%3Etail%20%3D%20newcell%3B%20%20%0A%20%20%20%20%20%20%7D%0A%20%20%20%20%7D%0A%20%20%7D%0A%7D%0A%0Aint%20main%28%29%20%7B%0A%20%20t_ht_list%20myhtlist%3B%0A%20%20myhtlist.head%20%3D%20myhtlist.tail%20%3D%20NULL%3B%0A%20%20insertOrderedHtList%28%26myhtlist,%205%29%3B%0A%20%20insertOrderedHtList%28%26myhtlist,%207%29%3B%0A%20%20insertOrderedHtList%28%26myhtlist,%204%29%3B%0A%20%20insertOrderedHtList%28%26myhtlist,%206%29%3B%0A%20%20insertOrderedHtList%28%26myhtlist,%2012%29%3B%0A%0A%0A%20%20return%200%3B%0A%7D&cumulative=false&curInstr=65&heapPrimitives=nevernest&mode=display&origin=opt-frontend.js&py=c_gcc9.3.0&rawInputLstJSON=%5B%5D&textReferences=false

Les listes chaînées circulaires

La dernière cellule pointe sur la première cellule

Liste chaînée circulaire

La liste chaînée circulaire (*circular linked list*) est une liste chaînée ne possédant pas de fin. En effet, le pointeur de fin de liste pointe vers le début de la liste formant ainsi un cycle.



Exemple de représentation d'une liste chaînée circulaire

On peut utiliser cette variante de la liste chaînée pour stocker par exemple le tour de chaque joueur dans un jeu, imaginons un jeu de carte qui se joue au tour par tour dans lequel plusieurs joueurs participent, une liste chaînée circulaire permettrait de stocker l'ordre de jeu des joueurs facilement.

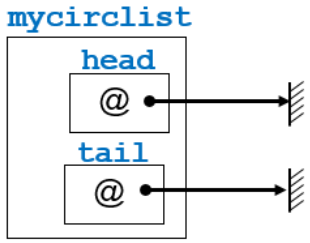
Tiré de : https://haltode.fr/algo/structure/liste_chainee.html

Définition de la structure

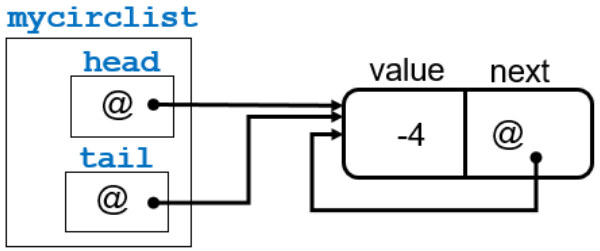
Une liste chaînée circulaire, dont le type est **t_circ_list**, n'est rien de plus qu'une liste 'tête et queue' **t_ht_list** (puisqu'on stocke l'adresse de la première cellule et que la dernière doit la pointer)

Il faut juste faire attention à ce que la dernière cellule pointe sur la première, au lieu de pointer sur NULL

Une liste chaînée circulaire vide :



Une liste chaînée circulaire avec un élément :



On se contente, pour définir ce type, de : **typedef t_ht_list t_circ_list;**

Une liste simple ne suffirait pas ?

Dans l'absolu, si

Le pointeur `'tail'`, pour repérer la fin de liste, pointe sur la dernière cellule, comme le fait l'avant dernière
`'tail'` duplique donc une information **qui est déjà** dans la liste

Mais.....

Imaginons une liste circulaire de 50000 éléments

Je veux insérer une nouvelle cellule en tête : c'est simple...mais la première cellule a changé !

Et la dernière doit pointer sur la première

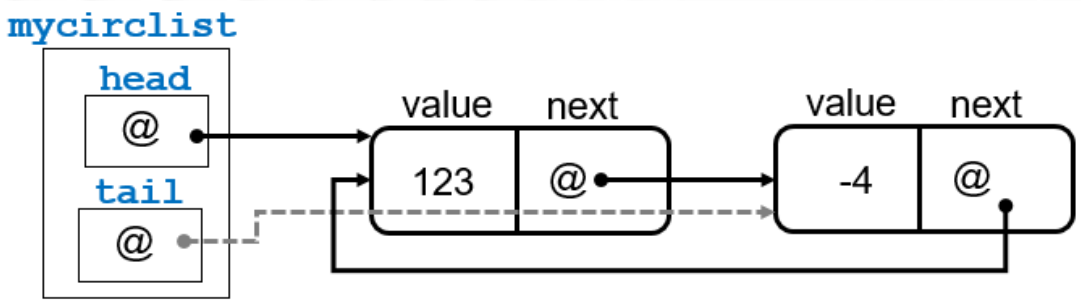
Donc je dois mettre à jour la valeur de la dernière

Donc je dois m'y déplacer (et parcourir toute la liste)

`'tail'` stocke justement cette information, pour ne pas reparcourir toute la liste

Illustration

Une liste chaînée circulaire avec plusieurs éléments



Question : qu'est-ce qui va changer, par rapport à une `t_list` ou `t_ht_list` ?

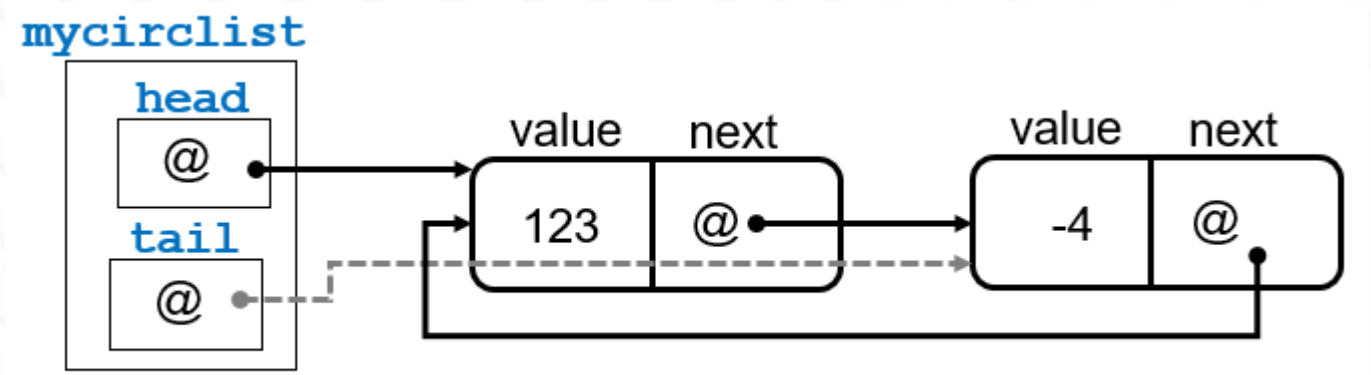
Réponse : la condition 'la liste se termine par NULL' – une liste circulaire n'a pas de fin !

Condition beaucoup utilisée !

Quelle est la nouvelle condition de fin ?

Réponse : **La cellule suivante est la première cellule**

Illustration



Soient :
`mycirclist` une liste circulaire (on la suppose remplie)
`temp` un pointeur de cellule (défini par `t_cell *temp`) qui se déplace dans la liste

La condition `temp == NULL` devient, dans la plupart des cas,

```
temp->next == mycirclist.head  // la suivante est la première
```

Chaînage en tête

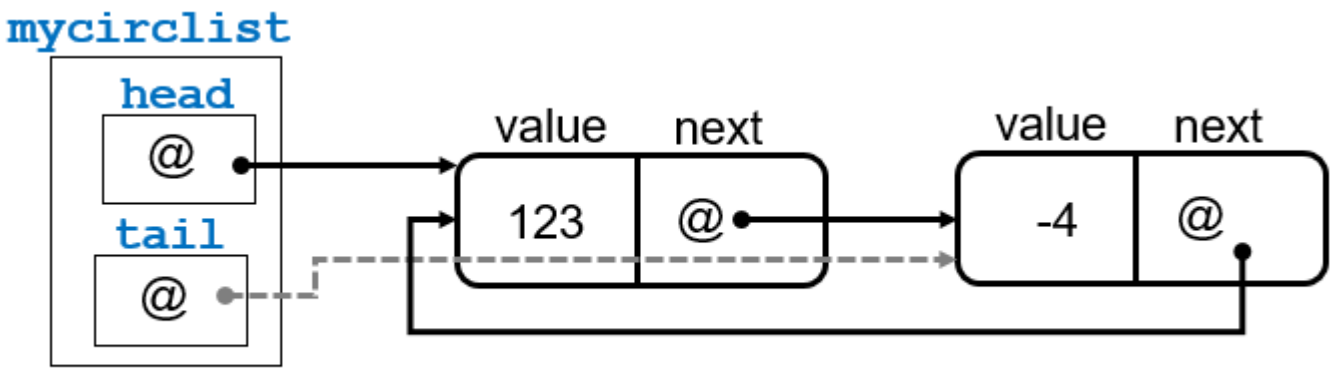
Qu'est ce qui change ?

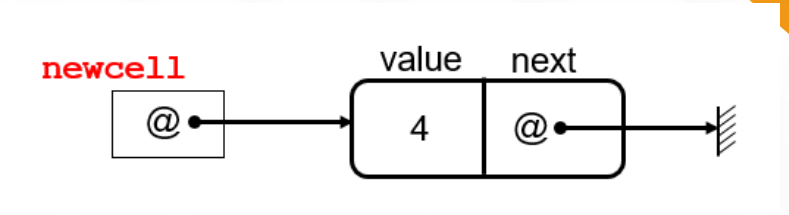
Si la liste n'est pas vide, on ajoute la première cellule en tête, on **met à jour la dernière pour qu'elle pointe sur cette nouvelle cellule**

Si la liste est vide, la première cellule qui y est insérée est aussi la dernière : elle doit pointer 'sur elle-même'. `'head'` et `'tail'` pointent cette nouvelle cellule

Rappel : `'head'` indique où est la première
`'tail'` indique où est la dernière

Cas d'une liste non vide





Dans l'ordre :

La suivante de la nouvelle est celle qui est actuellement première

```
newcell->next = mycirclist.head;
```

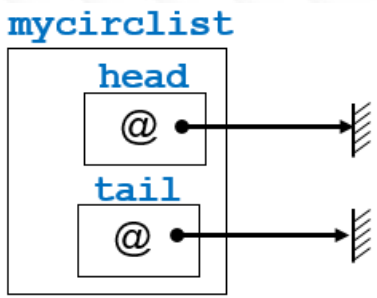
La première devient la nouvelle

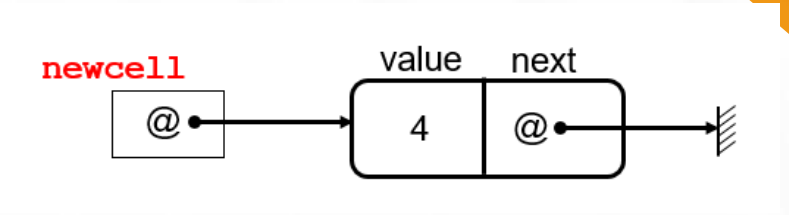
```
mycirclist.head = newcell;
```

La dernière pointe sur la nouvelle, qui est passée en première position

```
mycirclist.tail->next = newcell;
```

Cas d'une liste vide





Dans l'ordre :

La suivante de la nouvelle est elle-même, elle est première et dernière

```
newcell->next = newcell
```

La première devient la nouvelle

```
mycirclist.head = newcell
```

La dernière devient la nouvelle

```
mycirclist.tail = newcell
```

Chaînage en queue

Vu en TD/TP

Supprimer une cellule dans une liste circulaire

Il faut maintenir la cohérence de la liste :

Le cas général est quand la cellule n'est ni la première, ni la dernière
on parcourt la liste avec deux pointeurs, `temp` et `prev` (supprimer une cellule = sa précédente pointe sur sa suivante)

Si la liste est vide, on ne fait rien

Sinon, on cherche la cellule concernée, si elle existe

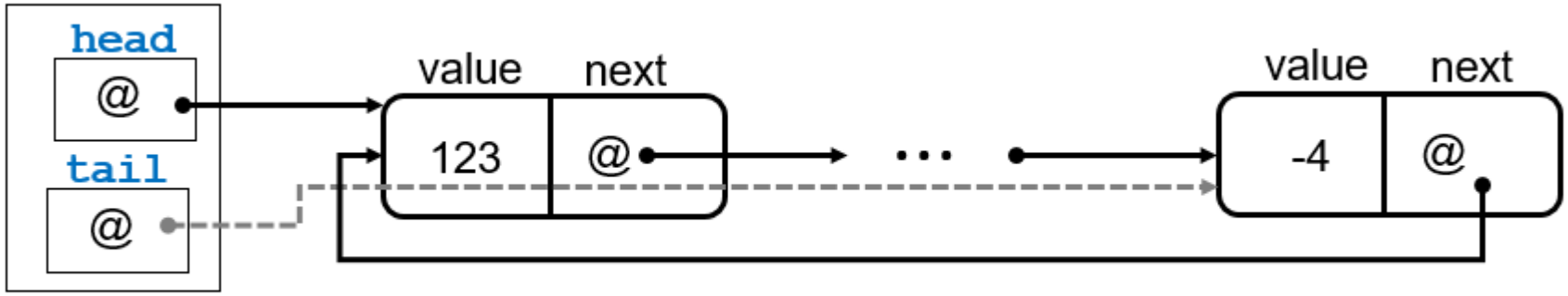
Si elle n'existe pas, on ne fait rien

Si elle existe, on a plusieurs cas à traiter !!

Supprimer une cellule dans une liste circulaire

On suppose qu'on a trouvé la cellule à supprimer

Cas numéro 1 : elle est en tête de liste



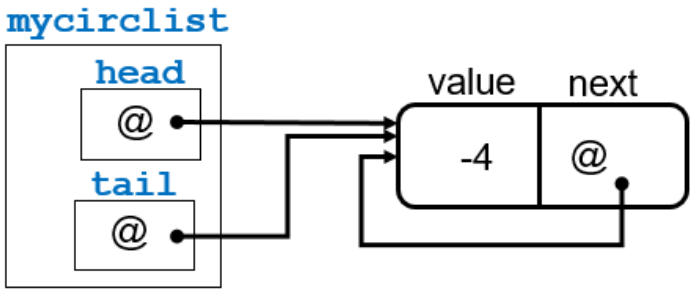
1) La nouvelle première est la suivante de l'actuelle première que l'on veut supprimer
2) La dernière pointe sur cette nouvelle première
3) (optionnel) On détruit l'ancienne première

Supprimer une cellule dans une liste circulaire

Peut-on appliquer cette méthode pour une liste avec une seule cellule ?

1) La nouvelle première est la suivante de l'actuelle première que l'on veut supprimer
2) La dernière pointe sur cette nouvelle première
3) (optionnel) On détruit l'ancienne première

À :



La suivante de l'actuelle première est elle-même !

1) Ne change rien
2) Ne change rien
3) Si on le fait, on détruit la cellule pointée par `'head'` et `'tail'` !

Supprimer l'unique cellule d'une liste circulaire

Comment détecter qu'il y a une seule cellule ?

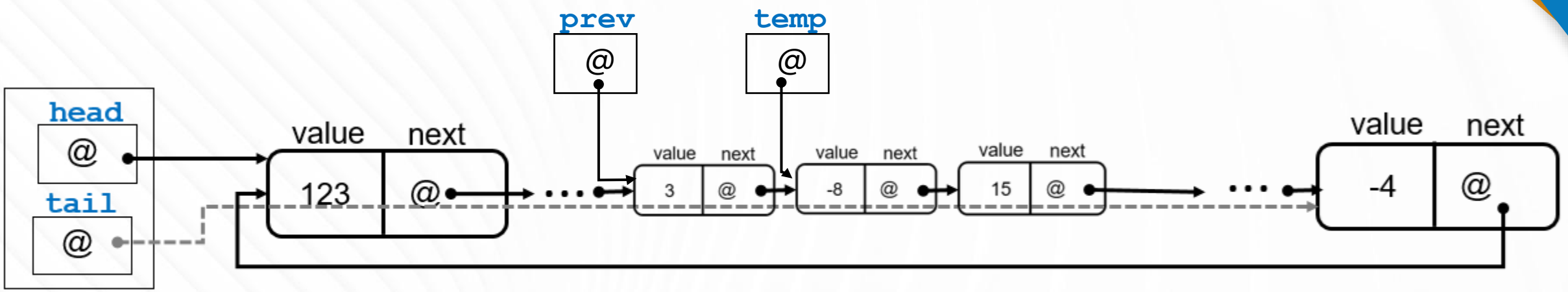
Indice : elle est la première et la dernière

Réponse : les pointeurs `'head'` et `'tail'` sont égaux

Pour la supprimer, on remet `'head'` et `'tail'` à NULL (**après** avoir détruit la cellule si on le décide)

Supprimer une autre cellule de la liste

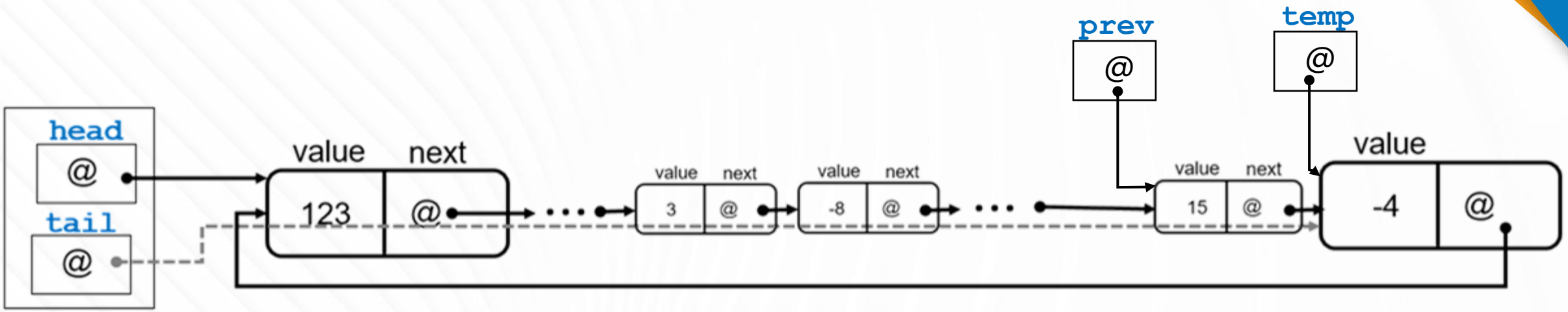
On suppose que le pointeur temp pointe sur la cellule à supprimer et que le pointeur **prev** pointe sur la cellule précédente



La suivante de **prev** devient la suivante de **temp**

Supprimer la dernière cellule

La suivante de `prev` devient la suivante de `temp`



<u>Et l'avant dernière devient la dernière :</u>
Il faut mettre à jour `'tail'` qui pointait au même endroit que `temp`, elle pointe maintenant sur la précédente de `temp`, qui est `prev`.

Les listes doublement chaînées

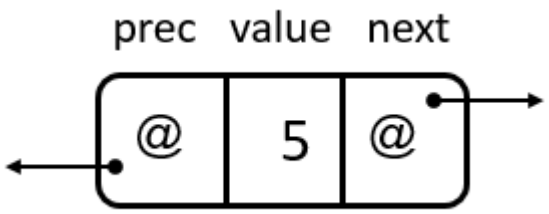
Chaque cellule pointe sur la suivante et la précédente

Cela nous oblige à définir une nouvelle structure, car c'est un nouveau type de cellule !

```
struct s_dcell
{
        struct s_dcell *prec;
        int value;
        struct s_dcell *next;
};

typedef struct s_dcell t_dcell;
```

Représentation :



La structure de liste associée

Comme pour les listes simples, on crée une structure avec un champ qui stocke l'adresse d'une cellule 'double'

```
struct s_dbl_list
{
        t_dcell *head;
};

typedef struct s_dbl_list t_dbl_list;
```

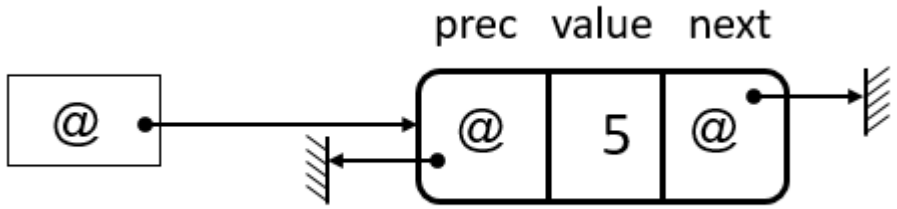
Création d'une cellule 'double'

De la même manière que l'on crée une cellule 'normale' avec la fonction `createCell()`, on crée une cellule double en initialisant sa valeur et ses deux pointeurs.

La fonction `createDCell()` doit donc recevoir en paramètre un entier (pour initialiser la valeur), et retourner l'adresse d'une cellule double stockant cette valeur.

Puisque c'est une unique cellule, elle n'a ni suivante : `next` est `NULL`, ni précédente : `prec` est `NULL`

Si, par exemple, on dema eur -7, on doit obtenir :

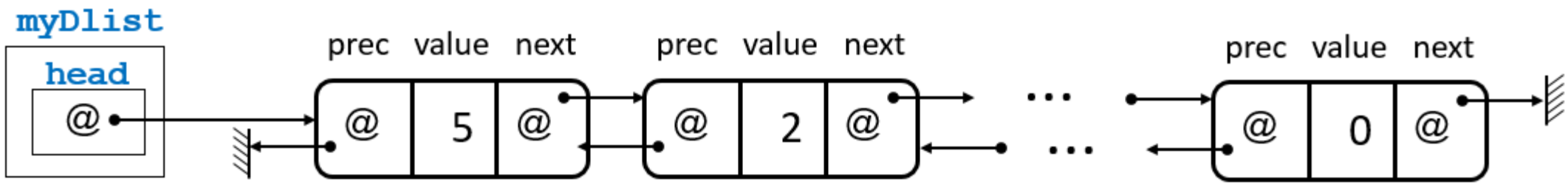


Cette fonction sera écrite en TD/TP

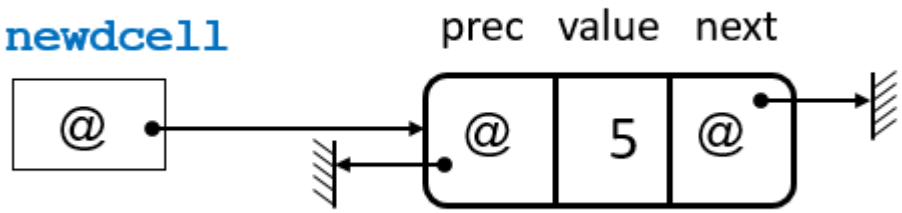
Insertion d'une cellule double en tête de liste

Prenons le cas général d'une liste double nommé `myDlist` et qui contient déjà des éléments

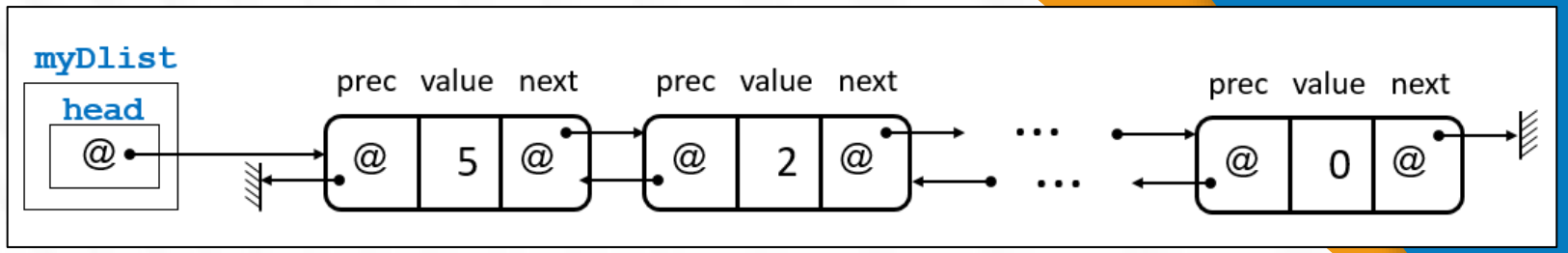
Sa représentation est :



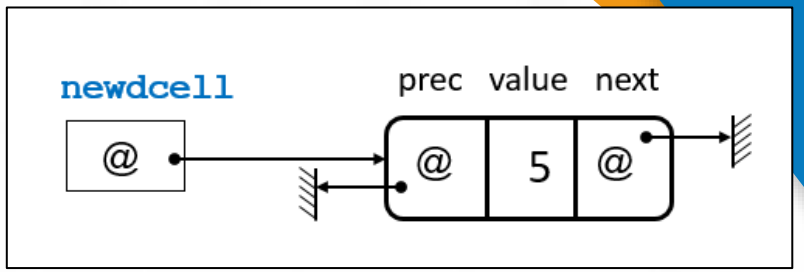
On souhaite insérer en tête une nouvelle cellule pointée par `newdcell`



Suite



Quelles instructions utiliser ?



Presque comme pour une liste simple :

La suivante de la nouvelle est l'actuelle première

```
newdcell->next = myDlist.head;
```

La précédente de l'actuelle première est la nouvelle

```
myDlist.head->prec = newcell;
```

la nouvelle devient la tête de liste

```
myDlist.head = newdcell;
```

A voir en exercice de TD pour la version finale

Insertion et suppression dans une liste double

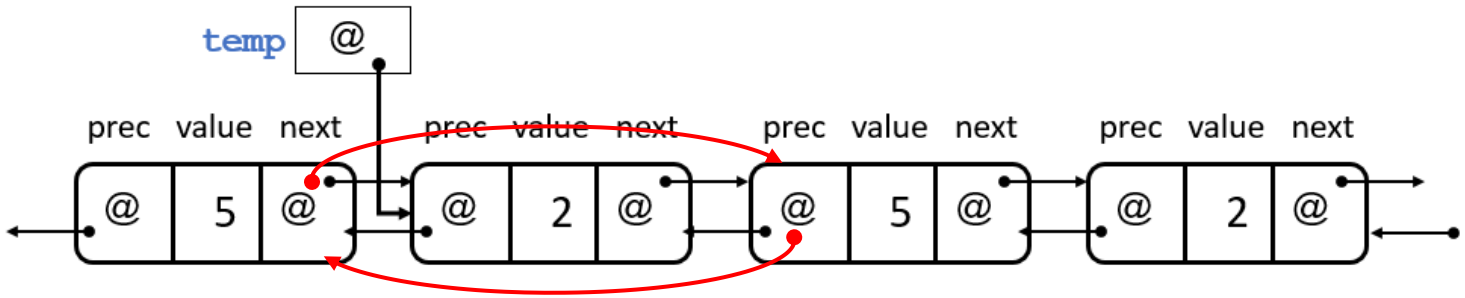
Rappel : dans les liste simples, ht, et circulaires, **on ne peut pas aller ‘en arrière’**

Il est nécessaire d'utiliser un pointeur `'temp'` pour repérer la cellule courante et un pointeur `'prev'` pour repérer la cellule précédente.

Ici, chaque cellule indique où est la précédente

Schémas sur les slides suivantes :

Schéma : cas général de suppression



Pour la flèche rouge de gauche à droite (chaînage classique)
La suivante de la précédente de temp devient la suivante de temp

```
temp->prec->next = temp->next
```

(temp->prec joue le rôle de 'prev')

Pour la flèche rouge de droite à gauche (chaînage arrière)
La précédente de la suivante de temp devient la précédente de temp

```
temp->next->prec = temp->prec
```

Cela ne fonctionne que si temp->prec n'est pas NULL (donc pas en tête)
Cela ne fonctionne que si temp->next n'est pas NULL (donc pas en fin)

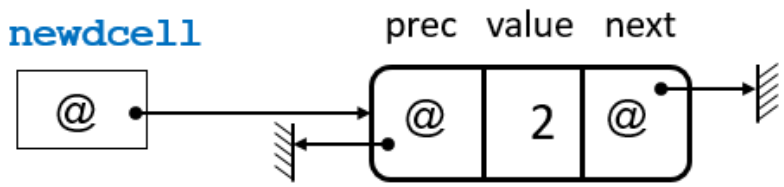
Suppression en tête et en queue

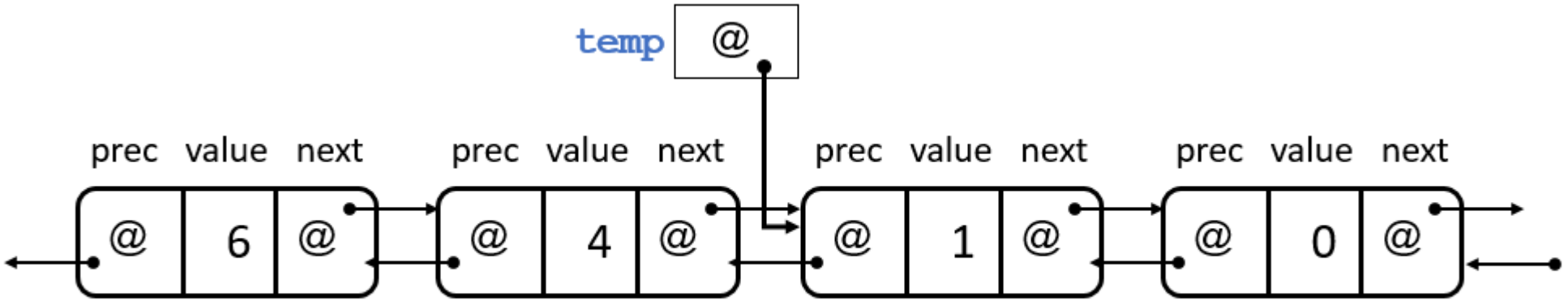
Seront vus en TD/TP

Schéma : cas général d'insertion

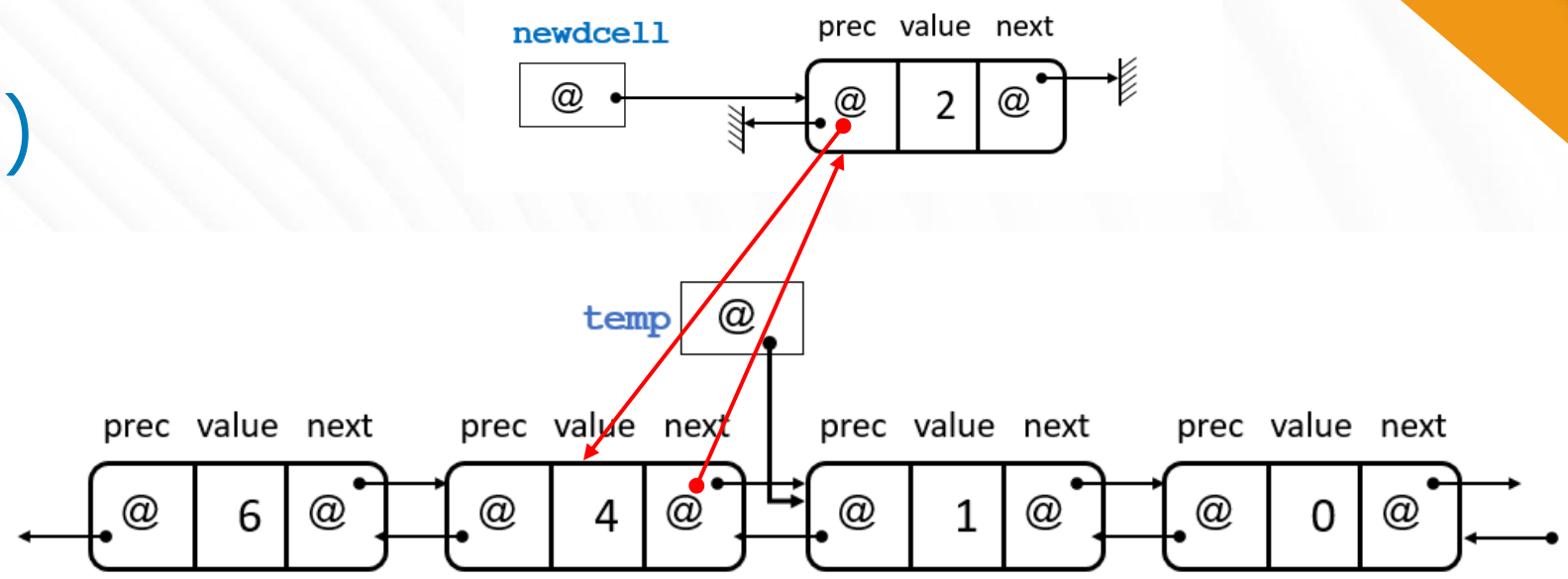
Par exemple, on doit insérer une cellule double dans une liste classée par ordre décroissant.

Pour rappel, dans ce cas classique, une cellule 'trop loin' dans la liste? Ici, on veut insérer la cellule newdcell stockant





Insertion (2)



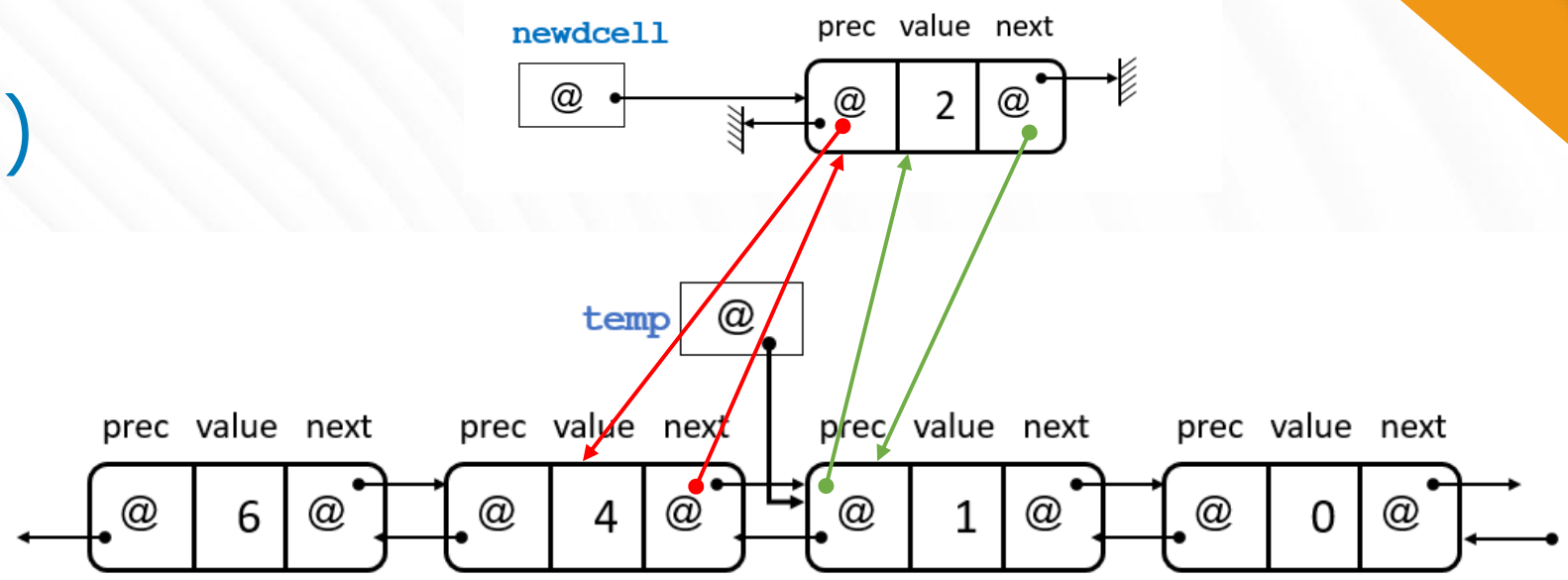
Pour la flèche rouge de gauche à droite (chaînage classique)
La suivante de la précédente de temp devient la nouvelle

```
temp->prec->next = newdcell  ;
```

Pour la flèche rouge de droite à gauche (chaînage arrière)
La précédente de la nouvelle devient la précédente de temp

```
newdcell->prec = temp->prec;
```

Insertion (3)



Pour la flèche verte de gauche à droite (chaînage classique)
La suivante de la nouvelle est la cellule courante temp

```
newdcell->next = temp;
```

Pour la flèche verte de droite à gauche (chaînage arrière)
La précédente de temp est la nouvelle

```
temp->prec = newdcell;
```

Insertion en tête et en queue

Seront vus en TD/TP